

Übungsblatt 3

Vorlesung „Secure Software Systems“ Wintersemester 2025/26

Praktische Übung Code-Reuse 1

Return-into-libc: Unterschiede zwischen x86 32-Bit und 64-Bit

In dieser Aufgabe werden Sie einen klassischen Code-Reuse Angriff durchführen: Einen Return-into-libc (ret2libc) Angriff. Das Ziel ist es mit Hilfe der libc-Funktionen eine Shell zu starten, um dann beliebige weitere Befehle auszuführen. Dabei sollen Sie die Unterschiede zwischen ret2libc auf x86 32-Bit und 64-Bit kennenlernen. Dazu analysieren Sie zwei fast identische Programme, die aus dem gleichen Source Code (`bufovf.c`) gebaut wurden. `bufovf-32` wurde jedoch für x86 32-Bit kompiliert und `bufovf-64` für x86 64-Bit. Entwickeln Sie einen *Return-Into-Libc* (ret2libc) Angriff, der zuerst die Funktion `system` aufruft um eine Shell zu starten und dann das Programm mittels `exit` sauber beendet.

Aufgabe 1.

- i) Übernehmen Sie zuerst die Kontrolle über den Stack mit Hilfe der Buffer Overflow Schwachstelle. Überschreiben Sie das Instruction Pointer Register (EIP bzw. RIP) mit einem kontrollierten Wert (z.B. `0x41414141`). Welchen Input müssen Sie dazu verwenden? Wie lang ist dieser Input? Was sind hier die Unterschiede zwischen den 32- und 64-Bit Varianten?
- ii) *32-Bit*: Bauen Sie den Stack so auf, dass ein Return-into-libc Angriff zu Stande kommt. Verwenden Sie die `system` Funktion zum Erstellen einer Betriebssystem-Shell und verwenden Sie die `exit` Funktion, um das Programm danach sauber zu beenden. Als Parameter für `system` übergeben Sie den String, der den Pfad zur Shell enthält. Als Parameter für `exit` übergeben Sie den Status Code `0x42`. Implementieren Sie den Angriff in `exploit-32.py`.
- iii) *64-Bit*: Führen Sie den gleichen Angriff, wie in der vorhergehenden Aufgabe beschrieben, nun auf x86 64-Bit durch. Da hier die Parameter über Register übergeben werden, müssen Sie ROP Gadgets verwenden, um die Register zu setzen. Verwenden Sie die vorgegebenen ROP Gadgets und implementieren Sie den Angriff in `exploit-64.py`.

Function Reuse und ROP (x86-64)

In dieser Aufgabe implementieren Sie einen Code-Reuse Angriff auf x86-64. Das Ziel des Angriffs ist es die Datei `flag.txt` auszugeben, um zu demonstrieren, dass durch den Angriff beliebige und potentiell sensible Dateien ausgelesen werden können. Auf x86-64 werden Argumente für Funktionsaufrufe bekanntlich nicht auf dem Stack, sondern in Registern übergeben. In dieser Aufgabe muss daher ROP verwendet werden, um die Register zu setzen. Weiterhin müssen mehrere Funktionen nacheinander ausgeführt werden um die Datei auszugeben, wofür wiederum ROP benötigt wird. Dabei dürfen nur die ROP Gadgets benutzt werden, die von uns in das Zielfunktionprogramm eingefügt wurden.

Um die Datei `flag.txt` auszulesen, müssen Sie die zwei Funktionen `open_file` und `head` kombinieren. Rufen Sie zuerst `open_file` auf und verwenden Sie den Rückgabewert als letzten Parameter für die Funktion `head`.

Aufgabe 2.

- i) Identifizieren und erklären Sie die nötigen Gadgets, um die Parameter von `open_file` und `head` zu setzen. In welchen Registern werden welche Parameter übergeben und welche Gadgets verwenden Sie zum setzen der Parameter? Finden und erklären Sie die Gadgets, die es erlauben den Rückgabewert von `open_file` als 3. Parameter von `head` zu verwenden.
- ii) Erweitern Sie `exploit.py` zu einem funktionsfähigen Exploit. Der Exploit soll zuerst `open_file` mit den richtigen Parametern aufrufen, um die Datei `flag.txt` zu öffnen. Verwenden Sie danach die Funktion `head`, um die Datei auszugeben.

ROP und *Unintended Instructions Sequences* (x86-64)

Das Programm `rop2shell` ist mittels eines Stack-basierten Buffer Overflows verwundbar, ansonsten hat das Programm keine Funktionalität. Es befinden sich jedoch einige ROP Gadgets in dem Programm. Das Ziel dieser Aufgabe ist es, eine Betriebssystem Shell zu starten. Dies zeigt, dass die Aufgabe eines klassischen Shellcodes auch mittels ROP realisiert werden kann. Da die Adresse der libc wegen ASLR nicht bekannt ist, kann kein return-into-libc Angriff durchgeführt werden.

In dieser Aufgabe sollen Sie einen einfachen ROP-Gadget-Finder implementieren. Dabei suchen Sie im ausführbaren Speicherbereich nach `ret` Instruktionen und disassemblieren rückwärts von dieser Instruktion aus. Analysieren Sie für alle so gefundenen Gadgets, in welcher Funktion diese vorkommen und ob diese eventuell Teil einer sogenannten *unintended instruction sequence* sind. Nutzen Sie *Cutter* oder *objdump* (z.B. mit dem Befehl `objdump -M intel -d getreal2 | less`), um reguläre Instruktionssequenzen zu finden.

Aufgabe 3.

- i) Überschreiben Sie die Return-Adresse zuerst mit dem Wert `0x4141414141414141`, um zu demonstrieren, dass Sie den Instruction Pointer übernehmen können. Wo im Programm ist die Schwachstelle? Welchen Input müssen Sie dem Programm schicken? Wie lang ist der Input?
- ii) Identifizieren Sie mögliche Gadgets, die Sie für die ROP-Chain nutzen können. Suchen Sie an den im Exploit-Template vorgegebenen Adressen nach möglichen Gadgets. Implementieren Sie dazu einen einfachen *ROP-Gadget-Finder*. Untersuchen Sie dazu den Assembler Code im Bereich des Symbols `gadgets` und der Funktion `compute` (dies ist im `exploit.py` Template bereits vorgegeben).
- iii) Welche der Gadgets können Sie verwenden, um die benötigten Register für den Systemcall `execve` zu setzen?
- iv) Erklären Sie „Unintended Instruction Sequences“ im Kontext von ROP auf x86. Eine solche Sequenz befindet sich unter den gegebenen ROP-Gadgets und enthält ein Gadget, welches Sie für einen erfolgreichen Angriff benötigen. Beschreiben Sie:
 - Welches Gadget haben Sie gefunden?
 - Warum ist das gefundene Gadget eine unintended instruction sequence?
 - Wie haben Sie es gefunden?
- v) Finden Sie eine Möglichkeit, um einen Systemcall als Teil Ihres ROP-Angriffes auszuführen. Welche Instruktion(en) bzw. ROP-Gadgets können Sie verwenden, um den Systemcall durchzuführen? Erklären Sie den Assembler Code und den Zweck der Gadgets.
- vi) Vervollständigen Sie `exploit.py` zu einem funktionsfähigen Exploit, der mit ROP eine Shell mittels des `execve` Systemcalls startet.